Palantir Foundry Developer Field Manual

USAREUR-AF Operational Data Team — Maven Smart System Context

Version 1.0 | Compiled March 2026

TABLE OF CONTENTS

1. Platform Overview & Architecture
2. Maven Smart System (MSS)
3. Getting Started as a Developer
4. The Ontology
5. Object & Link Types
6. Actions & Workflows
7. Functions on Objects (FOO)
8. Code Repositories & Python Transforms
9. Incremental Transforms
10. Pipeline Builder (Visual ETL)
11. Data Integration & Connectivity
12. Datasets — Storage, Transactions & Branches
13. Data Lineage
14. Object Storage Backend
15. Workshop (Application Builder)
16. Slate (Custom Applications)
17. Quiver & Contour (Analytics)
18. Code Workspaces & Model Training
19. AIP — AI Platform
20. AIP Logic
21. AIP Agent Studio
22. Ontology SDK (OSDK)

23. Foundry Platform SDK (Python)
24. Permissions, Roles & Access Control
25. Markings & Classification-Based Access Controls (CBAC)
26. Foundry Branching & CI/CD
27. Best Practices
28. Training & Certification Resources
29. Key URLs & Reference Index

---

1. PLATFORM OVERVIEW & ARCHITECTURE

What Is Palantir Foundry?

Palantir Foundry is an enterprise data integration and operational intelligence platform. It is not a traditional data warehouse or BI tool — it is a full-stack platform that moves data from ingestion through transformation, semantic modeling, AI/ML, and operational application delivery on a single unified stack.

The core architectural thesis: connect raw data to real-world decisions without fracturing existing sources of truth or forcing data duplication.

Two-Layer Architecture

Data Layer Handles all ETL/ELT operations, pipeline orchestration, storage, and compute. Everything from raw source connectors through cleaned, enriched, and curated datasets lives here.

Operational Layer (The Ontology) Bridges technical data infrastructure and operational end-users. The Ontology models real-world entities as objects with properties, relationships, actions, and business logic. This layer is what enables non-technical operators to work directly with data through applications.

Core Platform Components

Component	Role
Data Connection	Source connectors, syncs, agents

Component	Role
Pipeline Builder	Visual ETL / no-code transforms
Code Repositories	PySpark/SQL/Java transforms, functions
The Ontology	Semantic model: objects, links, actions, functions
Object Storage V2	Indexes and serves object data
Workshop	Drag-and-drop operational app builder
Slate	HTML/CSS/JS custom application builder
Contour	No-code tabular data analysis
Quiver	Ontology-native analysis and dashboards
AIP	AI Platform: LLMs, Logic, Agent Studio
Code Workspaces	JupyterLab / RStudio for ML development
OSDK	Ontology SDK for external application development
Data Lineage	Visual pipeline graph and dependency tracker
Foundry Branching	End-to-end development/staging environment

Key Foundry Concepts at a Glance

- **RID (Resource Identifier)**: Every Foundry resource (dataset, object type, function, etc.) has a unique RID. Format: `ri.foundry.main.dataset.<uuid>`.
- **Project**: The primary security boundary and organizational unit. Contains files, datasets, and collaborators.
- **Branch**: An isolated copy of the development environment for safe development and testing before merging to Main.
- **Transaction**: A versioned write operation against a dataset (SNAPSHOT, APPEND, UPDATE, DELETE).
- **Marking**: A mandatory access control that restricts data visibility — users must hold the marking to access marked resources.

---

2. MAVEN SMART SYSTEM (MSS)

Background

Project Maven was established in 2017 as DoD's flagship AI initiative, originally focused on computer vision for ISR (Intelligence, Surveillance, Reconnaissance) analysis. It transitioned to NGA in 2023 as a Program of Record.

Maven Smart System (MSS) is Palantir's implementation, purpose-built for Combined Joint All Domain Command and Control (CJADC2). It uses Palantir Foundry as the underlying data and AI platform.

Contracts & Scope

- DEVCOM Army Research Laboratory awarded Palantir a contract (up to $99.8M over 5 years) to expand MSS access across Army, Air Force, Space Force, Navy, and USMC.
- An earlier 5-year contract expanded access to thousands of users across five Combatant Commands: CENTCOM, EUCOM, INDOPACOM, NORTHCOM/NORAD, and TRANSCOM.

Current Deployments (as of early 2026)

MSS has production-level deployments at: - INDOPACOM - **EUCOM** (covers USAREUR-AF AOR) - CENTCOM - NORAD/NORTHCOM - SPACECOM - TRANSCOM - AFRICOM - Joint Staff - CYBERCOM (additional) - NATO SHAPE (acquired separately in 2025)

Foundry Tools Within MSS

MSS presents data geospatially and analytically using several integrated tools built on Foundry:

Tool	Function
Foundry	Core data platform: pipelines, ontology, applications
Gaia	Geospatial intelligence and visualization layer
Target Workbench	AI-assisted targeting workflow application
Maverick	AI/ML-powered pattern-of-life and threat analysis

Tool	Function
LogX	Logistics and sustainment analytics

Technical Contributions of MSS

- **Data fusion via Ontology**: MSS standardizes heterogeneous ISR, logistics, and operational data through the Foundry Ontology layer. This enables cross-domain data fusion, exchange across cloud and edge systems, and multi-application reuse of the same data.
- **AI-enabled battlespace awareness**: Rapid assessment, data aggregation, and prioritization of targets using ML models.
- **Contested logistics**: Supply chain visibility and disruption response.
- **Joint fires and targeting workflows**: AI-assisted target identification and engagement sequencing.
- **Force management**: Personnel and equipment readiness tracking.

EUCOM / USAREUR-AF Context

EUCOM (and subordinate commands including USAREUR-AF) operates MSS in the European theater. Relevant operational implications for developers in this AOR:

1. **Data sensitivity**: All operational data in EUCOM MSS deployments is classified or CUI at minimum. CBAC markings are mandatory (see Section 25).
2. **Ontology schemas**: Object types representing military units, equipment, positions, events, and targeting data are defined centrally — local teams may extend but not arbitrarily redefine core schemas.
3. **Allied interoperability**: NATO's acquisition of MSS (SHAPE, April 2025) means potential interoperability requirements with NATO C2 systems; data schemas may need to accommodate NATO standards (e.g., APP-6 symbology, JC3IEDM/MIM data models).
4. **Edge considerations**: Foundry supports edge deployments for disconnected/degraded environments — relevant for forward-deployed units.

NATO Integration

In April 2025, NATO acquired MSS through SHAPE for military planning. The JWC (Joint Warfare Centre) has been training to integrate MSS into NATO exercises, using its geospatial data presentation and AI planning tools.

3. GETTING STARTED AS A DEVELOPER

Access Pathways

Option 1 — Organizational Access If your organization already uses Palantir (e.g., USAREUR-AF MSS deployment), contact your Foundry Administrator to be provisioned an account with appropriate roles and markings.

Option 2 — AIP Developer Tier (Unclassified Training) Palantir offers a free Developer Tier for individuals to learn and build. Access at: `build.palantir.com`. Suitable for training and familiarization only — not for operational data.

Option 3 — AIP Bootcamp Structured onboarding program organized by Palantir for enterprise customers.

Role Taxonomy in MSS/Foundry

Role	Description	Tools Used
Data Engineer	Builds pipelines, manages ingestion, transforms raw data	Data Connection, Pipeline Builder, Code Repositories
Ontology Builder	Designs object types, link types, action types, properties	Ontology Manager
Application Developer	Builds operational apps for end-users	Workshop, Quiver, Slate
Data Scientist / ML Engineer	Trains and deploys models	Code Workspaces, Model Studio, AIP
Frontend / OSDK Developer	Builds external apps using the Ontology SDK	OSDK TypeScript/Python/Java repos
AIP Developer	Builds AI-powered workflows and agents	AIP Logic, Agent Studio
Platform Admin	Manages users, groups, markings, roles	Control Panel, Settings

Developer Progression Path

```
New User
  → Consume data in Contour/Quiver
  → Build analyses and dashboards
  → Build Workshop applications (Application Developer)
  → Write Python/SQL transforms (Data Engineer)
  → Build ontology schemas (Ontology Builder)
  → Write TypeScript/Python Functions (FOO Developer)
  → Build AIP Logic / Agents (AIP Developer)
  → Build OSDK applications (Frontend Developer)
```

Key Starting Resources

- `learn.palantir.com` — Official training portal (requires Foundry access)
- **Speedrun: Your First End-to-End Workflow** — Recommended first course; builds a complete workflow in under 60 minutes
- **Workshop Design Hub** — Marketplace product with example apps and templates
- **Build with AIP** (`build.palantir.com`) — Pre-built reference products and starter packs
- **AIP Assist** — In-platform LLM assistant for navigation and code generation

Platform Navigation Essentials

- Access to Foundry is via your enrollment URL (e.g., `https://your-org.palantirfoundry.com`)
- The **navigation side panel** provides access to all applications
- **Projects** are the primary organizational unit — request access to relevant projects from your Administrator
- **Data Lineage** is your primary tool for understanding pipeline dependencies

---

4. THE ONTOLOGY

Conceptual Definition

The Palantir Ontology is the semantic and operational layer that sits above raw datasets. It is both:

1. **A digital twin** of your organization — encoding real-world entities, their properties, relationships, and business rules
2. **An operational nervous system** — connecting data to decisions through actions and AI-powered functions

Dataset-to-Ontology mapping:

Datasets	Ontology Equivalent
Dataset	Object Type
Row	Object (instance)
Column	Property
Cell value	Property Value
JOIN between datasets	Link Type
JOIN result row	Link (instance)

Ontology Components

Semantic Layer (Structure) - **Object Types**: Schema definitions of real-world entities (e.g., `Soldier`, `Vehicle`, `MissionEvent`, `SupplyRequest`) - **Properties**: Attributes of object types (e.g., `dodID`, `unitAssignment`, `lastKnownPosition`, `status`) - **Link Types**: Relationship schemas between object types (e.g., `Soldier → assignedTo → Unit`) - **Shared Properties**: Reusable property definitions across multiple object types - **Interfaces**: Define polymorphic shapes shared across multiple object types - **Structs**: Complex nested data types within properties

Kinetic Layer (Operations) - **Action Types**: Defined operations users can perform (e.g., `UpdateSoldierStatus`, `SubmitMissionReport`, `EscalateAlert`) - **Functions**: Arbitrary business logic written in TypeScript or Python, attached to the ontology - **Object Views**: Rich, configured detail views for individual objects

Security Layer (Dynamic) - **Mandatory Control Properties**: Properties containing markings/organizations/classifications that drive row-level security policies

Ontology Architecture — How Data Flows In

```
External Data Source
      ↓
Data Connection (Sync)
      ↓
Raw Dataset (in Foundry)
      ↓
Pipeline Builder / Code Repositories (Transforms)
      ↓
Clean/Enriched Dataset
      ↓
Ontology Manager (Object Type backed by Dataset)
      ↓
Object Type (indexed in Object Storage V2)
      ↓
Workshop / Quiver / OSDK Applications
```

Object Explorer

The Object Explorer application allows any user with appropriate permissions to search, filter, and browse objects across all object types in the Ontology. It is the primary "catalog" view of ontological data.

Object Views

Object Views are the central information hub for a single object instance. They can display: - Object properties (formatted, conditional) - Linked objects (traversing link types) - Embedded Workshop modules - Charts and time series - Action buttons - Related files and media

---

5. OBJECT & LINK TYPES

Object Type Configuration

Each object type requires: - **Primary Key**: Uniquely identifies each object instance. Supported base types: `String`, `Integer`, `Short`. - **Backing Datasource**: A Foundry dataset (or restricted view, stream, etc.) that provides the raw data for objects. - **Properties**: Mapped from dataset columns to semantic property definitions.

Property Base Types

Type	Notes
`String`	Valid as primary key and title key
`Integer`	Valid as primary key
`Short`	Valid as primary key
`Long`	Discouraged as primary key
`Boolean`	—
`Byte`	—
`Float`	Not valid as primary key
`Double`	Not valid as primary key
`Decimal`	Not valid as primary key
`Date`	Discouraged as primary key (uniqueness issues)
`Timestamp`	Discouraged as primary key
`Array`	Cannot contain nulls; valid as title key
`Struct`	No nesting; no array fields
`Vector`	Cannot be title or primary key
`Geopoint`	Valid as title key only
`Geoshape`	Restricted use
`Media Reference`	Complex type
`Time Series`	Complex type
`Attachment`	Complex type
`Marking`	Specialized — drives mandatory control policies
`Cipher`	Specialized

Special Property Types

Derived Properties: Computed from other properties, not stored in the backing dataset.

Function-Backed Properties: Properties whose values are computed on-the-fly by a TypeScript or Python Function. Example: computing a threat score from multiple raw properties.

Mandatory Control Properties: Properties of type `Marking`, `Organization`, or `Classification`. Used to implement row-level security — objects are only visible to users who satisfy the control values on each object instance.

Shared Properties: Property definitions reused across multiple object types, ensuring semantic consistency (e.g., a `location` property with standardized format used on `Vehicle`, `Base`, and `MissionEvent`).

Property Reducers: Aggregation functions applied to linked objects' properties (e.g., count of linked incidents).

Configuring an Object Type — Steps

1. Navigate to **Ontology Manager**
2. Create or select an existing Object Type
3. Assign a **Backing Datasource** (dataset or restricted view)
4. Define the **Primary Key** mapping
5. Map dataset columns to **Properties** (using automapping or manual)
6. Configure **formatting** (display format, conditional formatting)
7. Set up **Link Types** to other object types
8. Configure **Action Types** for writeback
9. Set **Object-level security policies** if row-level security is required
10. **Index** the object type (triggers compute to build the object database)

Link Types

Definition: A link type defines a relationship schema between two object types. A link is one instance of that relationship.

Cardinality options: - One-to-one (1:1) - One-to-many (1:N) - Many-to-many (M:N) — requires a dedicated backing datasource on the link type itself (a "junction" dataset)

Self-referential links: An object type can have a link type to itself (e.g., `Soldier → reportsTo → Soldier` for chain-of-command).

Cross-Ontology limitation: Links cannot span multiple Ontologies. Use shared Ontologies for cross-org relationships.

Link backing: For 1:N and M:N links, the link is backed by a column on the "many" side's dataset (foreign key). For M:N, a separate junction dataset (with both IDs) backs the link type.

Interfaces

Interfaces define a common shape that multiple object types can implement. Enables polymorphic queries — you can query all objects implementing the `Asset` interface regardless of whether they are `Vehicle`, `Aircraft`, or `Weapon` object types.

Key capability: `Only TypeScript v2 supports the ability to access and edit Ontology interfaces.`

---

6. ACTIONS & WORKFLOWS

What Are Action Types?

Action types define the set of changes users can make to the Ontology in a single atomic operation. They are the primary write-back mechanism — connecting operational decisions back into the data platform.

Example action types in a military context: - `SubmitSITREP` — creates a new SITREP object - `UpdateSoldierReadinessStatus` — modifies a property on a Soldier object - `AssignMissionToUnit` — creates a link between a Mission and a Unit object - `EscalateIncident` — modifies status property AND creates a notification side effect - `ArchiveAsset` — soft-deletes by setting a status property

Action Type Components

Parameters: Inputs the user provides when executing the action. Configurable with: - Default values - Dropdown filtering (filtering the selectable objects/values) - Free-text input, date

pickers, numeric inputs, boolean toggles - Object references (select an existing object as a parameter)

Rules: Logic governing how parameter values translate to ontology changes. A rule may: - Set a property value - Create a new object - Create or delete a link - Apply a conditional transformation

Submission Criteria: Validation that must pass before the action can be submitted. Prevents bad data entry.

Side Effects: Actions triggered upon successful commit: - **Notifications**: Alert specified users or groups - **Webhooks**: Call an external endpoint (useful for integration with Army systems, ticketing, etc.)

Function-Backed Actions: Complex actions whose logic is computed by a TypeScript or Python Function rather than simple rule-based configuration. Enables batch execution (applying an action to multiple objects at once).

Writeback Architecture

When a user executes an action: 1. Changes are validated against submission criteria 2. Rules are applied, producing a diff 3. The diff is committed to the **writeback dataset** associated with the object type 4. Object Storage V2 indexes the changes 5. The updated objects are immediately visible in all downstream applications 6. Side effects (notifications, webhooks) fire asynchronously

The writeback dataset is a Foundry dataset that acts as the source of truth for user-generated edits, separate from the pipeline-generated backing dataset.

Inline Edits

Workshop and Object Views support "inline edits" — clicking directly on a property value and editing it in place, backed by an action type without requiring a separate form/modal.

Actions in Branch Environments

When testing actions on a Foundry Branch: - Webhooks and email notifications are **disabled** - Actions backed by external system integrations **cannot run** - All modified object types must be indexed on the branch before action testing

---

7. FUNCTIONS ON OBJECTS (FOO)

Overview

Functions are code-based business logic attached to the Ontology. They accept Ontology objects/object sets as inputs and can: - Compute derived values (function-backed properties) - Execute complex aggregations - Drive action logic (function-backed actions) - Power custom behaviors in Workshop and Quiver

Supported Languages

Language	Key Capabilities
TypeScript v2	Full Ontology access, OSDK support, interfaces, deployed execution, Node.js modules, up to 8 vCPUs/5GB RAM
TypeScript v1	Basic ontology access, webhook support, serverless only
Python	Pipeline Builder integration, local development, deployed execution, custom aggregations
AIP Logic	No-code LLM-powered functions (see Section 20)

Recommendation: Use TypeScript v2 for all new function development. Python for data-heavy functions integrated into pipeline workflows.

TypeScript v2 Function Structure

```
import { Function, Integer, Timestamp } from "@foundry/functions-api";
import { Soldier, MissionEvent } from "@foundry/ontology-api";

// Basic function: compute days since last deployment
export class SoldierFunctions {

    @Function()
    public daysSinceLastDeployment(soldier: Soldier): Integer | undefined {
        if (soldier.lastDeploymentDate == null) {
            return undefined;
        }
        const now = new Date();
```

```
        const last = new Date(soldier.lastDeploymentDate);
        const diffMs = now.getTime() - last.getTime();
        return Math.floor(diffMs / (1000 * 60 * 60 * 24));
    }
}
```

Accessing Object Properties

Properties are accessed via dot notation. All property accesses may return `undefined` — handle explicitly:

```
const firstName = employee.firstName;           // may be undefined
const city = `${airport.city}, ${airport.country}`;  // template literal
```

Traversing Links

Single-cardinality links (1:1, M:1):

```
const manager = employee.manager.get();         // may return undefined
const managerAsync = await employee.manager.getAsync();
```

Multi-cardinality links (1:N, M:N):

```
const subordinates = employee.reports.all();    // returns array (empty if none
const subAsync = await employee.reports.allAsync();
```

ObjectSet Operations (Efficient Traversal)

For large-scale traversal without loading object instances into memory:

```
// Search around from an object set to linked objects
const linkedObjects = soldierObjectSet.searchAroundToUnit();

// Filter object sets
const activeSoldiers = Soldier.where(s => s.status.exactMatch("ACTIVE"));
```

Array Properties

```
// Array properties return ReadOnlyArray — copy before modifying
let tags = [...asset.tags];
tags.push("NEW_TAG");
```

Importing Ontology Types

All object and link types must be imported into the Function repository's project before use:

```
import { Soldier, Unit, MissionEvent } from "@foundry/ontology-api";
// Or for private ontologies:
import { Asset } from "@foundry/ontology-api/your-ontology-name";
```

Function Testing

Functions support a **Live Preview** mode: import backing datasources, select specific objects, and execute the function to see real results in the development environment.

---

8. CODE REPOSITORIES & PYTHON TRANSFORMS

Overview

Code Repositories is the version-controlled development environment within Foundry for writing: - Python transforms (PySpark, Pandas, Polars, DuckDB) - Java transforms (Spark) - SQL transforms - R transforms - Container-based transforms - TypeScript/Python Functions (FOO) - ML model training code

Repositories are Git-backed (internally) with Foundry's branching model layered on top.

Python Transform Project Structure

```
transforms-python/
├── build.gradle              # Gradle build config; controls CI checks, plugins
├── conda/
```

```
│   └── conda_recipe.yml      # Python dependencies and environment definition
├── src/
│   └── myproject/
│       ├── __init__.py
│       ├── datasets/
│       │   └── my_transform.py    # Transform definitions
│       └── pipelines/
│           └── my_pipeline.py     # Pipeline registrations
└── test/
    └── myproject/
        └── test_my_transform.py   # Pytest unit tests
```

Core Transform Decorators

`@transform` — Raw transform with full access to file system and metadata:

```
from transforms.api import transform, Input, Output

@transform(
    my_output=Output("/path/to/output/dataset"),
    my_input=Input("/path/to/input/dataset"),
)
def my_transform(my_input, my_output):
    df = my_input.dataframe()         # Returns a Spark DataFrame
    # ... transform logic ...
    my_output.write_dataframe(df)
```

`@transform_df` — Simplified DataFrame transform (automatically reads/writes DataFrames):

```
from transforms.api import transform_df, Input, Output
from pyspark.sql import functions as F

@transform_df(
    Output("/path/to/output"),
    my_input=Input("/path/to/input"),
)
def compute(my_input):
    return my_input.filter(F.col("status") == "ACTIVE") \
                    .withColumn("processed_at", F.current_timestamp())
```

`@transform_pandas` — Pandas-based transform for smaller datasets:

```
from transforms.api import transform_pandas, Input, Output
import pandas as pd

@transform_pandas(
    Output("/path/to/output"),
    input_df=Input("/path/to/input"),
)
def compute(input_df: pd.DataFrame) -> pd.DataFrame:
    return input_df[input_df["status"] == "ACTIVE"].copy()
```

Pipeline Registration

Transforms are organized into Pipelines that define execution order and dependency graphs:

```
from transforms.api import Pipeline
from myproject.datasets.my_transform import my_transform

my_pipeline = Pipeline()
my_pipeline.discover_transforms(my_transform)
```

Key APIs on TransformInput

```
input_dataset.dataframe()              # Read as Spark DataFrame
input_dataset.pandas()                 # Read as Pandas DataFrame
input_dataset.filesystem()             # Access raw files (FileSystem API)
input_dataset.branch_name()            # Current branch name
input_dataset.schema()                 # Dataset schema
```

Key APIs on TransformOutput

```
output_dataset.write_dataframe(df)     # Write Spark DataFrame
output_dataset.write_pandas(df)        # Write Pandas DataFrame
output_dataset.filesystem()            # Write raw files
output_dataset.set_mode("REPLACE")     # or "MODIFY" for incremental
```

FileSystem API (Raw File Access)

For transforms that process non-tabular data (JSON, binary, media):

```
@transform(
    output=Output("/path/to/output"),
    raw_input=Input("/path/to/raw/files"),
)
def process_files(raw_input, output):
    with raw_input.filesystem().open("myfile.json", "r") as f:
        data = json.load(f)
    with output.filesystem().open("result.json", "w") as f:
        json.dump(processed_data, f)
```

Compute Options

Compute Engine	Use Case
Spark (PySpark)	Large-scale distributed processing
Pandas	Small to medium datasets, complex Python logic
Polars	High-performance Python DataFrame operations
DuckDB	SQL-like operations with pushdown optimization
Virtual Tables	Query-time compute against external warehouses (Snowflake, BigQuery, Databricks)
Bring-Your-Own-Container	Custom environments, specific dependencies

Spark Profiles

Apply a compute profile in `build.gradle` or via the repository UI to control executor size, driver memory, and parallelism. Managed profiles are recommended for most workloads — Palantir auto-tunes them. Custom profiles for large-scale or GPU workloads.

```
// build.gradle example — apply linting plugins
apply plugin: 'com.palantir.conda.pep8'
apply plugin: 'com.palantir.conda.pylint'
apply plugin: 'com.palantir.transforms.lang.antipattern-linter'
```

CI/CD Checks in Code Repositories

Checks run automatically on every commit via Gradle: - **PEP8 / Pylint** style checks (Python) - **Spark anti-pattern linter** (catches inefficient Spark patterns) - **Pytest unit tests** (see Section 8 — Unit Testing) - **Custom Gradle task checks** (team-defined validation rules) - **Impact analysis** (identifies downstream dependencies affected by changes)

Enable pytest in `build.gradle`:

```
apply plugin: 'com.palantir.transforms.lang.test-python'
```

Tests are auto-discovered by pytest using the `test_` prefix convention.

Local Development

Two options: 1. **Palantir extension for VS Code** (recommended) — provides integrated debugging, dataset preview, and transform execution from VS Code 2. **Gradle + local Python** — clone the repository, run `./gradlew :transforms-python:compileAll` locally

Supported Languages Summary

- Python (PySpark and native Python)
- Java (Spark)
- SQL (Spark SQL)
- R (via containerized execution)
- Container transforms (arbitrary language via Docker)

---

9. INCREMENTAL TRANSFORMS

Overview

Incremental transforms process only new or changed data since the last successful build, rather than reprocessing the entire dataset. Critical for performance in large-scale operational pipelines.

Transaction Types

Type	Behavior
SNAPSHOT	Replace entire output with fresh data. Simplest. No history of prior state.
APPEND	Add new rows only. Output grows over time. Prior data not modified.
UPDATE	Modify existing rows and add new ones. Requires stable primary keys.
DELETE	Remove records from output.

Append-Only Incremental (Most Common Pattern)

```
from transforms.api import transform, incremental, Input, Output

@incremental(semantic_version=1)
@transform(
    output=Output("/path/to/output"),
    new_data=Input("/path/to/source"),
)
def compute(new_data, output):
    # By default:
    #   - read_mode = "added" (reads only rows added since last run)
    #   - write_mode = "modify" (appends new rows to output)
    df = new_data.dataframe()
    output.write_dataframe(df)
```

Incremental with Full-Refresh Fallback

```
from transforms.api import transform, incremental, Input, Output

@incremental(semantic_version=1)
@transform(
    output=Output("/path/to/output"),
    source=Input("/path/to/source"),
)
def compute(source, output, ctx):
    if ctx.is_incremental:
        # Only process new rows
        df = source.dataframe(mode="added")
        output.write_dataframe(df, mode="modify")
    else:
        # Full refresh (first run or schema change)
```

```
        df = source.dataframe()
        output.write_dataframe(df, mode="replace")
```

Requirements for Incremental Execution

A transform runs incrementally only when ALL of the following are true: 1. All incremental inputs had only files ADDED (via APPEND or UPDATE transactions) since the last run 2. No schema changes have occurred to inputs 3. The `semantic_version` has not been bumped (bumping forces a full refresh)

If any condition is not met, the transform falls back to a full SNAPSHOT run.

Creating Historical Datasets from Snapshots

Foundry provides a pattern for building time-series history from snapshots — each new SNAPSHOT transaction becomes a row in the output with an associated timestamp, enabling reconstruction of state at any point in time.

---

10. PIPELINE BUILDER (VISUAL ETL)

Overview

Pipeline Builder is the no-code/low-code data integration application. It provides a graphical canvas where users build data pipelines by connecting nodes without writing code.

Target users: Data engineers who prefer visual development, analysts building one-off pipelines, teams needing rapid prototyping.

Pipeline Builder Workflow

1. **Add Inputs**: Connect existing Foundry datasets or create a new source connector
2. **Transform**: Apply transform nodes to filter, join, aggregate, pivot, etc.
3. **Preview**: Inspect output data at any node before committing
4. **Configure Outputs**: Write to datasets, object types, media sets, or virtual tables
5. **Schedule**: Set trigger conditions for automated execution

Transform Node Categories

Basic Operations - Filter rows (SQL-style condition builder) - Select / rename / reorder columns - Add column (expression-based) - Drop duplicates - Coalesce (null handling) - Limit rows

Join & Combine - Inner Join, Left Join, Right Join, Outer Join - Anti Join, Semi Join, Cross Join - Union (stack datasets vertically) - Geo distance join - Geo intersection join - KNN (K-Nearest Neighbors) spatial join

Aggregation & Reshape - Group by + aggregate (count, sum, avg, min, max, percentile, etc.) - Pivot / Unpivot - Window functions (rank, lead, lag, running totals)

File Parsing - CSV, JSON, GeoJSON, XML, Excel, Shapefile parsers - Compression handlers (gzip, tar, zip) - File metadata extraction

Advanced - K-means clustering - Pattern mining - OCR processing (extract text from images) - Media transformations - Time-bounded operations

Expression Functions (400+)

Pipeline Builder includes an expression language with 400+ built-in functions across:

- **Math**: `abs`, `ceil`, `floor`, `round`, `sqrt`, `log`, `power`, trigonometric functions
- **String**: `concat`, `substring`, `trim`, `lower`, `upper`, `regexp_extract`, `regexp_replace`, Base64 encode/decode
- **Date/Time**: `current_timestamp`, `date_add`, `date_diff`, `date_trunc`, `to_date`, `date_format`
- **Geospatial**: coordinate conversions, H3 indexing, geometry operations, WKT/WKB parsing
- **Array/Map**: `array_contains`, `array_explode`, `map_keys`, `map_values`, `size`
- **Type Casting**: `cast`, `try_cast`, logical type operations
- **Conditional**: `case when`, `if`, `coalesce`, `nullif`

Output Types

Output Type	Description
Dataset	Standard Foundry tabular dataset
Object Type	Write directly to an ontology object type backing dataset

Output Type	Description
Media Set	Unstructured file collection
Virtual Table	Query-time computed view (no materialization)
Geotemporal Series	Time-indexed spatial data

Scheduling

- **Time-based triggers**: Cron expressions for scheduled execution
- **Upstream triggers**: Run when an input dataset is updated
- **Manual triggers**: Ad-hoc execution
- **Parameterized pipelines**: Pass runtime parameters at execution time

Compute Integration

- Native Spark execution
- **Databricks compute pushdown**: Run computations in a customer-managed Databricks cluster
- **Snowflake compute pushdown**: Push SQL logic to Snowflake
- **Native acceleration**: Columnar engine for compatible workloads

---

11. DATA INTEGRATION & CONNECTIVITY

Data Connection Application

Data Connection is the Foundry application for connecting to and synchronizing data from external systems.

Architecture Components: - **Agents**: On-premise software connectors that run in your network and securely extract data. Required for on-prem or air-gapped sources. - **Sources**: Connection configurations (credentials, hostnames, connection strings) - **Syncs**: Scheduled or triggered data transfers from source to Foundry datasets - **Exports**: Reverse sync — push data from Foundry to external systems - **Webhooks & Listeners**: Event-driven ingestion triggers

Available Connectors (200+)

Databases & Data Warehouses - PostgreSQL, MySQL, Oracle, SQL Server, Snowflake, Databricks, BigQuery, Redshift

Cloud Storage - Amazon S3, Azure Blob Storage (ABFS), Google Cloud Storage, SharePoint Online

File Systems & Protocols - SFTP, FTPS, HDFS, SMB/CIFS, local filesystem

Enterprise Systems - SAP (with HyperAuto for automated discovery), Salesforce, Microsoft Dynamics, Oracle ERP

APIs & SaaS - REST APIs (custom JDBC), GitHub, Jira, Slack, HubSpot

Messaging / Streaming - Apache Kafka, Google Pub/Sub

Custom - Custom JDBC sources for any JDBC-compatible database

Sync Types

Sync Type	Description
Full batch sync	Replace entire Foundry dataset on each run
Incremental batch sync	Append only new/changed records using watermarks
Streaming sync	Continuous real-time ingestion (via Kafka, Pub/Sub)
CDC (Change Data Capture)	Track row-level changes in source database
File batch sync	Ingest files from S3, SFTP, filesystem
Media set sync	Ingest unstructured files (images, documents, video)

HyperAuto (SAP SDDI)

Automated SAP data discovery and pipeline generation — scans SAP metadata and auto-generates Foundry pipelines for standard SAP tables. Reduces SAP integration from months to days.

Private Link Connectivity

Foundry supports VPC-peered connectivity via AWS PrivateLink, Azure Private Link, and GCP Private Service Connect. Required for classified or controlled environments.

---

12. DATASETS — STORAGE, TRANSACTIONS & BRANCHES

Dataset Fundamentals

A Foundry dataset is the core data abstraction — a wrapper around a collection of files stored in Foundry's backing distributed file system. Datasets provide: - Integrated permission management - Schema management - Version control via transactions - Branching (parallel development copies)

Dataset RID format: `ri.foundry.main.dataset.<uuid>`

Supported File Formats

Parquet (default for transforms), CSV, JSON, Avro, ORC, raw binary, media files.

Transaction Model

Every write to a dataset is a **transaction**. Transactions create an immutable audit log of all changes.

Transaction Type	Behavior
`SNAPSHOT`	Replaces the entire dataset with new data. The "view" resets to only this transaction's files.
`APPEND`	Adds new files to the current view. Does not modify existing files.
`UPDATE`	Modifies existing records. Requires primary key alignment.
`DELETE`	Removes records from the view.

Transaction API (v1)

Create Transaction:

```
POST /api/v1/datasets/{datasetRid}/transactions

Request body:
{
    "transactionType": "SNAPSHOT"  // or APPEND, UPDATE, DELETE
}

Query params:
    branchId: "main"  // optional, defaults to master

Response:
{
    "rid": "ri.foundry.main.transaction.<uuid>",
    "transactionType": "SNAPSHOT",
    "status": "OPEN",
    "createdTime": "2026-03-11T12:00:00Z"
}
```

Error codes: - `409 CONFLICT`: Open transaction already exists - `403 PERMISSION_DENIED`: Insufficient write permissions - `404 NOT_FOUND`: Dataset or branch not found

Dataset Views and Branches

A **dataset view** represents the effective file contents at a branch at a point in time. The view is computed as: 1. Start at the most recent `SNAPSHOT` transaction (or earliest if none exists) 2. Layer all subsequent `APPEND` / `UPDATE` / `DELETE` transactions on top

Branches: Every dataset can have multiple branches (e.g., `main`, `feature/my-branch`). Changes on a branch don't affect `main` until merged. This mirrors Git branching semantics.

Schema Management

Foundry datasets maintain schemas (column names + types). Schema changes trigger full-refresh of downstream incremental transforms. The API supports: - `GET /api/v2/datasets/{datasetRid}/schema` — retrieve schema - `PUT /api/v2/datasets/{datasetRid}/schema` — update schema

Dataset Preview

The Dataset Preview application in Foundry allows browsing dataset contents, schema, transaction history, and build logs without leaving the browser.

Iceberg Tables

Foundry supports Apache Iceberg table format — providing ACID transactions, schema evolution, time-travel queries, and partition pruning for high-performance analytical workloads.

---

13. DATA LINEAGE

Overview

Data Lineage is the interactive visualization tool for understanding how data flows through Foundry — from source through transforms to ontology objects to downstream applications.

Think of it as the "map" of your data pipelines.

Accessing Lineage

- Right-click any dataset → "View in Data Lineage"
- Access directly from the Data Lineage application
- Navigate upstream (trace where data comes from) and downstream (find what depends on this dataset)

Graph Node Types

Node Type	Description
Dataset	Foundry tabular datasets
Transform	Code Repository or Pipeline Builder transforms
Sync	Data Connection ingest operations
Ontology Entity	Object types backed by datasets
Schedule	Automated build triggers
Artifact	Packaged outputs (models, libraries)

Node Coloring

Nodes are color-coded to communicate status at a glance: - **Green**: Up to date, build successful - **Yellow/Orange**: Stale (upstream has changed; needs rebuild) - **Red**: Build failed - **Grey**: Not yet built / no build scheduled

Capabilities Within Data Lineage

- **Preview dataset**: View sample rows without leaving lineage view
- **Inspect transform logic**: View the code/config behind a transform node
- **Trigger builds**: Start a build directly from the lineage graph
- **Manage schedules**: Set or modify build schedules inline
- **Rollback**: Roll back a dataset to a prior transaction
- **Check permissions**: Verify your access level to any node
- **Save and share**: Generate a shareable read-only link to a lineage view
- **Export**: Export graph to SVG

Build Management from Lineage

The **Builds helper** panel (accessible within Lineage) allows starting targeted builds — choosing exactly which nodes to refresh and in what order. Useful for: - Partial rebuilds after a data hotfix - Forcing a full refresh on an incremental transform - Debugging stale data issues

---

14. OBJECT STORAGE BACKEND

Architecture

The object backend stores and serves indexed object data, separate from the raw dataset layer.

Object Storage V1 (Phonograph) — Legacy. Planned for EOL deprecation June 30, 2026. If your MSS environment uses V1, plan migration to V2.

Object Storage V2 — Current standard. Key improvements: - Supports up to **2,000 properties per object type** (vs. fewer in V1) - Incremental indexing by default (faster updates) - Horizontal scaling - Higher-capacity Search Arounds (default 100,000 objects; configurable higher) - Separates indexing and querying for independent scaling

Object Data Funnel

The Object Data Funnel is the pipeline that writes data into the object backend:

```
Backing Dataset (from transforms or actions)
        ↓
Object Data Funnel (orchestrator)
        ↓
Object Storage V2 (indexed database)
        ↓
Object Set Service (OSS) — query layer
        ↓
Applications (Workshop, Quiver, OSDK)
```

Object Sets

Object sets are collections of object instances. Types:

Object Set Type	Description
Static	Saved as a list of primary keys; does not update when new objects are created
Dynamic	Saved as filter criteria; updates automatically as data changes
Temporary	User-specific; expires within 24 hours
Permanent	Persisted in the object backend; accessible platform-wide

Object Set Service (OSS)

OSS handles all read operations: - Searching objects (full-text and property-based) - Filtering (property conditions) - Aggregating (count, sum, avg, etc.) - Search Arounds (traverse link types to find linked objects) - Loading object property values for display

---

15. WORKSHOP (APPLICATION BUILDER)

Overview

Workshop is Foundry's primary no-code/low-code operational application builder. It reads exclusively from the Object Data Layer (Ontology), enabling rich applications backed by the semantic model.

Three design principles: 1. **Object data foundation** — All data comes from object types and their relationships 2. **Consistent design system** — Enforced visual standards for operational clarity 3. **Interactive complexity** — Supports both dashboards and task-management workflows

Widget Categories

Core Display - Object Table, Object List, Object View, Property List - Links (display related objects), Object Set Title - Edit History, Data Freshness indicator

Visualization - Chart XY, Pie Chart, Gantt Chart - Map (geospatial), Timeline, Stepper - Vega Chart (custom D3-based visualizations) - Metric Card, Pivot Table - Spreadsheet Display, PDF Viewer - Media Preview, Video Display, Audio/Transcription Display - Image Annotation, Free-form Analysis, Time Series Analysis

Filtering - Filter List, Object Dropdown, Search Bar - String Selector, Date/Time Picker - Text Input, Numeric Input - User Select, Prominent Term, Exploration Filter Pills

Actions & Navigation - Button Group (triggers actions) - Inline Action (direct property edit) - Media Uploader, Audio Recorder - Comments, Tabs

AIP Integration - AIP Analyst (embed AI analysis) - AIP Agent (embed an AIP Agent) - AIP Generated Content

Embedding - iFrame (embed external URLs) - Embedded modules (embed other Workshop modules)

Variable System

Variables are the backbone of Workshop interactivity. Types:

Variable Type	Description
Object Set Filter Variable	Stores a dynamic filter on an object type; drives table/chart filtering
Struct Variable	Stores arbitrary structured data

Variable Type	Description
String / Number / Boolean Variable	Simple typed values
Module Interface Variable	Variables exposed as inputs to a module for cross-module communication

Variable transformations allow computing derived variables from others (e.g., aggregating a count from a filtered object set).

Events System

Events connect user interactions to state changes: - Button click → execute action → refresh object set - Row click in table → set selected object variable → update detail panel - Filter change → re-query object set → refresh chart

Workshop and Actions Integration

Buttons in Workshop are connected to Action Types. When a user clicks, the action form opens with configured parameters, the user fills it in, and submitting commits the change to the Ontology.

Advanced Workshop Features

- **Auto-refresh**: Configures automatic data polling on a schedule
- **Routing**: Multi-page navigation within a module
- **State saving**: Users can save their filter/view state
- **Scenario management**: Save, load, and apply "what-if" scenarios
- **Mobile support**: Specialized widgets and layout for mobile
- **Kiosk mode**: Full-screen operational display
- **Translations**: Multi-language support
- **Performance profiler**: Built-in tool to identify slow queries/widgets
- **Read-only dashboard mode**: Publish as a non-interactive display
- **App pairing**: Deep-link between Workshop modules

Accessing Workshop from Foundry Branching

Workshop modules can be developed on a branch. Data modified on that branch is available for testing within Workshop on the same branch. Actions run on branches do not fire webhooks or notifications.

---

16. SLATE (CUSTOM APPLICATIONS)

Overview

Slate is a more powerful (and more complex) application builder for cases where Workshop's widget library is insufficient. Slate allows full HTML, CSS, and JavaScript customization.

When to use Slate over Workshop: - You need pixel-perfect custom branding - You need a public-facing application (non-Foundry users) - You need to integrate external APIs and data sources beyond the Ontology - You need complex Handlebars templating logic - You need custom CSS animations or layout patterns

Slate Architecture

Data Layer - Ontology queries (object sets, individual objects) - Functions on Objects (FOO) — business logic - Actions (writeback) - Ontology SDK access - External APIs and databases (beyond Foundry)

Logic Layer - Variables (similar to Workshop but more flexible) - Handlebars templating for dynamic HTML - Event system - JavaScript functions for custom logic

Style Layer - Custom CSS per widget - Global stylesheets (experimental) - Full CSS flexibility (fonts, colors, borders, animations)

Public Applications

Slate uniquely supports deploying applications accessible on the public internet without requiring Foundry authentication. Use cases: - Citizen-facing data submission portals - Partner-facing status dashboards - Open data portals

Configuration: No server, DNS, or authentication setup required — Palantir manages the hosting.

Creating a Slate Application

1. Navigate to a Project → `+New → Slate application`
2. Provide: location, title, and permalink (unique URL identifier)
3. Configure pages (multi-page support)
4. Add widgets from the widget library
5. Connect data sources (Ontology queries, external APIs)
6. Apply custom CSS and HTML
7. Configure variables, events, and actions

---

17. QUIVER & CONTOUR (ANALYTICS)

Quiver

What it is: Point-and-click analysis tool for Ontology-native data — designed for users who work primarily with object types and time series.

Key capabilities: - Explore object type data without writing SQL or code - Leverage pre-mapped link types for relationship traversal (no manual JOINs) - Specialized time series analysis with sensor/signal processing functions - Custom formula language for derived metrics - Transform tables and materializations for complex aggregations - Writeback: capture analytical conclusions back to the Ontology

Best for: Operational analysts, intelligence analysts, anyone already working within the Ontology model.

Dashboard output: Quiver analyses become interactive dashboards embeddable in Workshop and Notepad reports.

Contour

What it is: Point-and-click analysis tool for raw tabular datasets — designed for dataset-level exploration before or outside the Ontology.

Key capabilities: - Filter, join, and transform tabular data visually - Build multi-step analytical paths with branching - Parameterize analyses to switch between views - Expression language

for advanced calculations (array functions, window functions, relative date derivation) - Save analysis output as new Foundry datasets - Create interactive dashboards from analyses

Best for: Data engineers validating pipeline outputs, analysts working with data not yet in the Ontology, ad-hoc data exploration.

When to use Contour vs. Quiver: | Scenario | Tool | |---|---| | Data not in Ontology | Contour | | Very large datasets (joins on 100k+ objects, agg on 50k+ rows) | Contour | | Need to save results as a new dataset | Contour | | Data is in the Ontology and you want to use links | Quiver | | Time series / sensor data analysis | Quiver | | Writeback to Ontology from analysis | Quiver |

---

18. CODE WORKSPACES & MODEL TRAINING

Overview

Code Workspaces provides browser-based interactive development environments (IDEs) for data science and ML model development within Foundry.

Available IDEs

IDE	Package Management
JupyterLab	Conda (`maestro env conda install`), PyPI (`maestro env pip install`)
RStudio Workbench	CRAN (`renv::install()`), Bioconductor (`renv::install("bioc::")`)

Accessing Foundry Data in Code Workspaces

1. Navigate to the **Data** tab in the workspace
2. Select `Add > Read data`
3. Search for the desired dataset
4. A code snippet is generated to read the dataset into a DataFrame

Note: Data imported from outside the workspace's current project requires a project reference.

Limitation: Dataset views cannot be directly imported — copy them to a dataset first.

Model Training Workflow

```
# 1. Read training data from Foundry
from foundry_ml import Model, Stage

df = spark.read.format("foundry").load("/path/to/training/dataset")

# 2. Train your model (sklearn, pytorch, etc.)
from sklearn.ensemble import RandomForestClassifier
clf = RandomForestClassifier()
clf.fit(X_train, y_train)

# 3. Wrap in a Foundry Model Adapter
class MyAdapter(Stage):
    def transform(self, df):
        predictions = self.sklearn_model.predict(df)
        return pd.DataFrame(predictions, columns=["prediction"])

# 4. Publish to Foundry Model Registry
model = Model(MyAdapter(clf))
model_reference.publish(model)
```

Model Adapter Requirements

Every model published to Foundry must be wrapped in a **Model Adapter** that defines: - `transform(df)` method accepting input data - Output schema definition - Any preprocessing/postprocessing logic

MLflow Integration

Foundry integrates with MLflow for experiment tracking. The **Experiments** feature in Foundry tracks: - Model hyperparameters - Training metrics across runs - Artifacts (model files, plots)

GPU Training

For GPU-requiring workloads (deep learning, LLMs): 1. Create a JupyterLab workspace 2. Configure GPU resources (1+ GPUs) at workspace creation 3. Use standard PyTorch/TensorFlow GPU training patterns 4. Publish the trained model to Foundry's model registry

Model Deployment

Published models are available for: - Inference in Pipeline Builder transform nodes - Scoring in Code Repository transforms - Real-time scoring via AIP Logic - Deployment as Model-backed Properties on object types

---

19. AIP — AI PLATFORM

Overview

Palantir AIP (Artificial Intelligence Platform) connects LLMs and ML models to operational data and workflows via the Foundry Ontology. It is not a standalone AI product — it is AI with organizational context, security, and governance baked in.

Core AIP builder tools: - **AIP Logic** — no-code LLM workflow builder - **AIP Agent Studio** — AI agent/assistant builder - **AIP Evals** — evaluation and testing framework for AI functions - **AIP Assist** — in-platform AI assistant for Foundry users

LLM Integration

Supported model types: - Palantir-hosted LLMs (via partnerships) - Customer-managed LLMs via Bring Your Own Model (BYOM) - LLM-provider compatible APIs (OpenAI-format)

Model management features: - LLM capacity management - Model version control - Deprecation tracking - User-level and function-level permissions on LLM access

Security Model

AIP inherits Foundry's full security stack: - LLMs only access data the requesting user is authorized to see - All LLM queries and responses are logged in Foundry's audit trail - Historical lineage tracking for all AI operations - No data leaves the platform to external LLM providers unless explicitly configured via BYOM

---

20. AIP LOGIC

Overview

AIP Logic is a **no-code development environment** for building functions powered by LLMs. Target users: anyone who wants to automate AI-driven workflows without writing TypeScript or Python.

What AIP Logic Functions Can Do

- Accept Ontology objects or free-text strings as inputs
- Query Ontology objects for context ("Query objects" block)
- Use LLM blocks to process text with engineered prompts
- Apply branching logic (conditional execution paths)
- Return structured outputs (objects and/or strings)
- Make Ontology edits (automatically apply or stage for human review)

Building an AIP Logic Function — Workflow

1. Open **AIP Logic** application
2. Define function inputs (object type fields, text parameters)
3. Add **"Query objects"** blocks to pull contextual Ontology data
4. Add **"Use LLM"** blocks with engineered prompts that reference inputs and queried objects
5. Add branching logic for conditional execution paths
6. Define outputs (text response, Ontology edit, object)
7. Configure intermediate parameters to evaluate block-level outputs
8. Test with sample inputs
9. Configure execution mode (manual trigger vs. automated via AIP Automate)
10. Deploy

Integration with AIP Automate

Logic functions can trigger Ontology edits that are: - **Automatically applied** (no human review required) - **Staged for human review** (creates a "proposed edit" that a user approves)

Compute Monitoring

AIP Logic includes dashboards for: - LLM token usage per function - Compute costs - Execution frequency and latency

AIP Evals

The evaluation framework for Logic functions: - Define evaluation suites with test cases (input → expected output) - Run experiments comparing different prompt configurations - Track metrics across evaluation runs - Write evaluation results back to Foundry datasets for analysis

---

21. AIP AGENT STUDIO

Overview

AIP Agent Studio builds interactive AI assistants (**AIP Agents**) equipped with enterprise-specific knowledge and tools. Agents are powered by LLMs + Ontology + documents + custom tools.

Agent Tier Framework

Tier	Description
Tier 1	AIP Threads — ad-hoc analysis, conversational data exploration
Tier 2	Task-specific agents with granular permissions and curated context
Tier 3	Agents integrated into Workshop modules or OSDK applications
Tier 4	Automated agents operating via AIP Automate workflows (no human in loop)

Context Retrieval Types

Agents use retrieval to provide LLMs with relevant context:

Context Type	Description
Ontology context	Live queries against object types — grounded in operational data
Document context	RAG over uploaded documents (SOPs, doctrine, manuals)
Custom function-backed context	Developer-defined retrieval logic (TypeScript/Python function)

Context Type	Description
Application variables	Dynamic filtered object sets passed from the surrounding Workshop app

Citations: Every LLM response is traceable to source documents or objects, enabling auditability.

Tool Integration

Agents can be equipped with tools that perform real actions: - Custom tools (developer-defined) - Commands-as-tools (execute pre-built Foundry commands) - Function interfaces for external system integration

Deployment Options

Deployment	Description
Workshop widget	Embed agent as an AIP Agent widget in a Workshop module
OSDK application	Access agent via Foundry APIs in external TypeScript/Python apps
Palantir Platform APIs	Programmatic access for custom integrations
Marketplace distribution	Share agents internally or with partner organizations

Building an Agent

1. Open **AIP Agent Studio**
2. Name and configure the agent (persona, system prompt)
3. Add retrieval context (Ontology queries, documents, functions)
4. Add tools (commands, custom functions)
5. Configure permissions (which objects/data the agent can access)
6. Set the agent tier / deployment target
7. Test with sample queries
8. Publish / deploy

Custom Retrieval Functions (Pro-Code)

For advanced use cases, developers can write TypeScript/Python functions that implement custom retrieval logic for agent context. This enables: - Hybrid search (semantic + keyword) - Custom ranking and filtering of context - Multi-hop reasoning patterns

---

22. ONTOLOGY SDK (OSDK)

Overview

The Ontology SDK (OSDK) enables developers to build applications **outside** of Foundry that use the Ontology as a backend. Think of it as Foundry as an API — your external React app, Python script, or Java service can query objects, traverse links, call functions, and execute actions.

Language Support

Language	Package Manager	Status
TypeScript / JavaScript	NPM	GA; TypeScript v2 recommended
Python	pip / conda	GA
Java	Maven	GA
Other	OpenAPI spec	Any language via code generation

Key Benefits

- **Type safety**: Generated types mirror your exact Ontology — property names and types are validated at compile time
- **Ergonomic APIs**: Read and write back to the Ontology with minimal boilerplate
- **Scoped access**: Applications use scoped tokens granting access only to required ontological entities
- **User permission inheritance**: Application users can only see data they are authorized to see
- **Metadata in editor**: Property descriptions and types are accessible via IntelliSense/autocomplete

TypeScript OSDK Setup

1. Create an application in **Developer Console**
2. Bootstrap using the TypeScript guide (generates package.json, auth config)
3. Install the generated OSDK package: `bash npm install @foundry/osdk-your-app-name`
4. Import and use types: ```typescript import { FoundryClient } from "@foundry/osdk-your-app-name"; import { Soldier, Unit } from "@foundry/osdk-your-app-name/ontology";

const client = new FoundryClient({ auth: / ... / });

// Query soldiers const activeSoldiers = await client.ontology.objects.Soldier .filter({ status: { eq: "ACTIVE" } }) .all(); ```

Python OSDK Setup

```
pip install foundry-osdk-your-app-name
```

```
from foundry_osdk import FoundryClient
from foundry_osdk.ontology import Soldier, Unit

client = FoundryClient(auth=token_auth, hostname="your-org.palantirfoundry.com"

# Query with filter
soldiers = client.ontology.objects.Soldier.filter(
    status="ACTIVE"
).page(page_size=50)
```

React Development

TypeScript OSDK provides React bindings for building Foundry-backed React applications. The Developer Console includes a VS Code workspace with React scaffolding pre-configured.

OSDK vs. Platform SDK

SDK	Purpose
OSDK	Query/modify Ontology objects, links, actions, and functions

SDK	Purpose
Platform SDK	Platform-level operations: manage datasets, users, groups, builds, schedules

---

23. FOUNDRY PLATFORM SDK (PYTHON)

Installation

```
pip install foundry-platform-sdk
```

Authentication

```
import foundry_sdk
import os

# Development (user token)
client = foundry_sdk.FoundryClient(
    auth=foundry_sdk.UserTokenAuth(os.environ["BEARER_TOKEN"]),
    hostname="example.palantirfoundry.com"
)

# Production (OAuth2 confidential client)
client = foundry_sdk.FoundryClient(
    auth=foundry_sdk.ConfidentialClientAuth(
        client_id=os.environ["CLIENT_ID"],
        client_secret=os.environ["CLIENT_SECRET"]
    ),
    hostname="example.palantirfoundry.com"
)
```

Environment variables: `FOUNDRY_TOKEN` and `FOUNDRY_HOSTNAME` auto-configure the client if set.

Key Operations

Dataset management:

```
# List branches of a dataset
for branch in client.datasets.Dataset.Branch.list(dataset_rid):
    print(branch.name)

# Create a transaction
txn = client.datasets.Dataset.Transaction.create(
    dataset_rid=dataset_rid,
    transaction_type="SNAPSHOT",
    branch_id="main"
)
```

Pagination (via ResourceIterator):

```
for item in client.datasets.Dataset.Branch.list(dataset_rid):
    print(item)  # automatically pages through all results
```

Error Handling

```
from foundry_sdk.exceptions import PalantirRPCException, NotFoundError

try:
    dataset = client.datasets.Dataset.get(rid)
except NotFoundError:
    print("Dataset not found")
except PalantirRPCException as e:
    print(f"API error: {e.status_code} - {e.message}")
```

Available Error Types

- `PalantirRPCException` (base)
- `NotFoundError`
- `BadRequestError`
- `UnauthorizedError`
- `ValidationError` (Pydantic type mismatches)

API Versioning

```
from foundry_sdk.v2 import FoundryClient  # explicit v2 (recommended)
```

Async Support (Beta)

```
from foundry_sdk import AsyncFoundryClient

async def main():
    async with AsyncFoundryClient(...) as client:
        branches = await client.datasets.Dataset.Branch.list(dataset_rid)
```

---

24. PERMISSIONS, ROLES & ACCESS CONTROL

Access Control Model Overview

Foundry uses a layered access control system:

```
Mandatory Controls (highest priority)
    → Organizations
    → Markings
    → Classification-Based Access Controls (CBAC)

Discretionary Controls (lower priority)
    → Roles (Owner, Editor, Viewer, Discoverer)
    → Project-level role grants
    → Resource-level role grants
```

Mandatory controls **always** override discretionary controls. A user with `Owner` role on a resource but lacking a required Marking cannot access that resource.

Default Role Hierarchy

Role	Power Level	Can Grant Roles Up To
Owner	Highest	Owner, Editor, Viewer, Discoverer
Editor	High	Editor, Viewer, Discoverer
Viewer	Medium	Viewer, Discoverer
Discoverer	Lowest	Discoverer only

What each role grants: - **Owner**: Full control — read, write, delete, manage permissions - **Editor**: Read, write, create derived resources; can grant up to Editor - **Viewer**: Read only; can see data and metadata - **Discoverer**: Can discover the existence of the resource (see its name/description) but cannot read data

Projects as Security Boundaries

Projects are the **primary security unit** in Foundry: - All resources within a project inherit the project's role grants - Recommended pattern: grant roles to **groups** at the **project level**, not to individuals at the resource level - Project Owners can disable file/folder-level role grants, enforcing project-level-only permission management

Cross-Project Access (References)

To use a dataset from Project A in Project B: 1. You must have **Viewer** (or higher) on Project A 2. You must have **Editor** (or higher) on Project B 3. Create a **reference** in Project B pointing to the dataset in Project A

The reference is a thin wrapper — the data remains in Project A; Project B accesses it through the reference while maintaining security boundaries.

Permission Request Workflow

Users without project access can submit access requests: 1. Navigate to the project 2. Click "Request Access" 3. Specify: required role, marking compliance claims, business justification 4. Request routes to Project Owners (or group administrators)

Object-Level Security

Beyond project/resource access, Foundry supports row-level and column-level security on object types:

Object Security Policies: Control which object instances a user can see, based on mandatory control property values on each object.

Property Security Policies: Control which property values within an object are visible, at the column level.

Access requires: 1. Viewer access to the object type 2. Passing the configured object security policy (row-level check) 3. Passing applicable property security policies (column-level check)

Failing a property policy returns `null` for that property; failing an object policy hides the entire object from the user.

Custom Roles

Default roles are customizable. Admins can create new roles with specific permission sets tailored to operational requirements. Managed in: `Foundry Settings → Roles` (requires Organization Administrator permission in Control Panel).

---

25. MARKINGS & CLASSIFICATION-BASED ACCESS CONTROLS (CBAC)

Markings Overview

Markings are **mandatory access controls** — unlike role-based permissions that grant access, markings restrict access by requiring users to hold specific marking memberships.

Key characteristics: - Binary (all-or-nothing): you either have the marking or you don't - Conjunctive: a user must satisfy ALL markings on a resource (AND logic) - Propagating: markings flow from Projects/folders to contained resources AND from upstream datasets to derived datasets

Marking Propagation

Markings propagate through two pathways:

1. **File hierarchy**: Marking on a Project or folder cascades to all resources inside
2. **Data dependency**: Markings on an upstream dataset travel with derived data through all downstream transforms

This means: if Dataset A is marked, Dataset B (derived from A) automatically inherits the marking — even if Dataset B has no explicit marking set.

Implication for developers: When you derive a dataset from classified data, the output inherits the classification. Users who lack the marking can see the output dataset exists (metadata) but cannot access the data.

Marking Configuration Strategies

Strategy	Description	Use Case
Per-data-category	One marking per classification type (e.g., CUI, SECRET, TS)	Broad data governance
Per-data-owner	Teams manage access to their own data assets	Decentralized governance
Pipeline-stage	Different markings for raw, processed, and refined stages	ETL access tiering

Example: Military Data Tiering

```
TOP SECRET data
    → Marking: TS_MARKING
    → Only users with TS clearance + TS_MARKING membership can see

SECRET data
    → Marking: SECRET_MARKING
    → Users need SECRET clearance + SECRET_MARKING membership

CUI / FOUO data
    → Marking: CUI_MARKING
    → Standard OPSEC-appropriate access control
```

Scoped Sessions

Users can enable **scoped sessions** — selecting a subset of their markings to work in a "compartmentalized" workspace, preventing accidental co-mingling of data at different classification levels.

Classification-Based Access Controls (CBAC)

CBAC provides hierarchical classification enforcement for sensitive government information. **Requires Palantir involvement to configure and is disabled by default.**

CBAC Dimensions (three independent layers):

Dimension	Controls
Project Classification	Resource discovery and project-level access
File Classification	Metadata visibility
Data Classification	Actual data viewing (auto-inherited from upstream dependencies)

Project Maximum Classification: Projects have a maximum classification ceiling. Resources with a higher classification cannot be created or moved into a lower-classified project.

Category Logic: - **Conjunctive category**: User must hold ALL markings in the category (AND logic) - **Disjunctive category**: User can access marked data by holding ANY one marking in the category (OR logic)

Enforcement Rule: `Project maximum classifications specify the maximum classification for all resources inside of a project.`

Mandatory Control Properties (Row-Level via Ontology)

For fine-grained, object-level access control:

1. Add a property of type `Marking` (or `Organization`, `Classification`) to an object type
2. Configure an **Object Security Policy** referencing that property
3. Objects are only visible to users who satisfy the mandatory control value on each individual object instance

Example: A `ThreatObject` with a `classificationLevel` property of type `Marking`. Users with `SECRET_MARKING` see SECRET-marked threats; users without it cannot see those rows at all.

---

26. FOUNDRY BRANCHING & CI/CD

Overview

Foundry Branching is the **end-to-end development and staging environment** — a unified branch that spans Code Repositories, Pipeline Builder, Ontology Manager, and Workshop

simultaneously. One branch covers your full stack.

This enables the critical workflow: develop transforms → develop ontology schema changes → build Workshop UI — all on the same branch, all testable together before merging to production.

Branch Lifecycle

```
Create Branch (from Main)
      ↓
Develop changes (transforms, ontology, Workshop)
      ↓
Index modified object types on branch
      ↓
Test (build pipelines, run actions, validate Workshop)
      ↓
Create a Proposal (equivalent to a Pull Request)
      ↓
Assign reviewers per resource type
      ↓
Approval cycle
      ↓
Merge (Deploy) to Main
      ↓
Branch archived/closed
```

Branch Statuses

Status	Meaning
Active	In progress or awaiting approval
Deployed	Merged to Main
Inactive	No activity after the retention period
Closed	No longer in use; resources deleted

Proposals (Pull Requests)

A Proposal contains: - Description of changes - List of modified resources (with diffs) - Assigned reviewers (per resource) - Approval status for each resource

Proposals can have three states: Active, Deployed, or Closed.

Supported Resources in Foundry Branching

Resource Type	Branching Support
Code Repository datasets	Yes
Pipeline Builder	Yes
Ontology (object types, link types, action types)	Yes
Workshop modules	Yes (only via Foundry Branching — no separate mechanism)
Quiver dashboards	No — cannot be tracked as branch resources
Views	Partial — can be built on branches via Data Lineage with "forced build"

Branch Constraints

- **Actions on branches**: Webhooks disabled; email notifications disabled; external system integrations cannot execute
- **Object types**: All modified object types must be indexed before testing actions
- **Workshop rebase**: If Main changes after your branch was created, use Workshop's rebase feature to highlight and resolve conflicts

Cost Management

- Every branch incurs compute and storage costs
- Large object type modifications are especially expensive
- **Retention policies** automatically close stagnant branches (configurable)
- Recommendation: keep branches short-lived; avoid large, long-running branches

Naming Conventions for Branches

Standard prefixes:

```
feature/my-new-feature
bugfix/fix-bad-join
hotfix/urgent-data-correction
jdoe/experimental-approach
```

Use the same branch name across all repositories within a product. This ensures downstream datasets read from the correct upstream branch.

CI/CD Checks Summary

Check Type	Location	Config
PEP8 style	Code Repo	`build.gradle`: `apply plugin: 'com.palantir.conda.pep8'`
Pylint	Code Repo	`apply plugin: 'com.palantir.conda.pylint'`
Spark anti-pattern linter	Code Repo	`apply plugin: 'com.palantir.transforms.lang.antipattern-linter'`
Pytest unit tests	Code Repo	`apply plugin: 'com.palantir.transforms.lang.test-python'`
Custom Gradle checks	Code Repo	Custom Gradle tasks in inner `build.gradle`
Data health checks	Pipeline Builder / Dataset	Configured expectations in Data Connection or PB
Foundry Linter	Platform-wide	Rules evaluate resource metadata; actionable recommendations
Impact analysis	Code Repo / PB	Identifies downstream dependencies before merge

Foundry Linter

The Foundry Linter (`/docs/foundry/linter/`) provides **platform-wide recommendations** (not just code): - Identifies pipelines that incur unnecessary costs - Flags resources not using

the latest platform features - Enforces organizational best practice rules - Helps optimize Ontology structure - Increases pipeline stability

---

27. BEST PRACTICES

Naming Conventions

General Rule: Prefer clarity over brevity. Two to three word names that orient the reader. Put the distinctive portion first (helps with truncated dropdowns).

```
GOOD: soldier_readiness_daily
GOOD: mission_events_raw
BAD:  soldierdata
BAD:  mr_d (abbreviations)
BAD:  the_dataset_that_tracks_soldier_readiness_daily_for_eucom (too long)
```

Columns and datasets: Use `snake_case` (never `camelCase` or `PascalCase` for data objects).

```
# GOOD
soldier_id, unit_assignment, last_known_position

# BAD
SoldierID, unitAssignment, LastKnownPosition
```

Repositories: Use `snake_case` or `kebab-case`, consistent with org convention.

Branches: Use prefixes: `feature/`, `bugfix/`, `hotfix/`, or `username/`.

Object Types: Use `PascalCase` for semantic clarity: `Soldier`, `MissionEvent`, `SupplyRequest`.

Action Types: Use verb-noun convention: `UpdateSoldierStatus`, `SubmitMissionReport`.

Project Structure Best Practices

- Keep Projects tightly scoped — one project per use case or data domain
- Avoid "feature creep" — don't add tangentially-related resources to an existing project

- Organize pipelines into logical layers: `raw/` → `staging/` → `curated/`
- Use a dedicated Project for shared/reusable libraries and schemas

Pipeline Development Best Practices

Idempotency: Transforms should produce the same output regardless of how many times they run with the same input. Use `SNAPSHOT` transactions or deduplicate on primary keys.

Incremental by default: For large, frequently-updated datasets, always implement incremental transforms. Avoid full reprocessing of unchanged data.

Partitioning: For datasets queried frequently with filters on a specific column (e.g., `event_date`, `unit_id`), configure Hive-style partitioning or dataset projections to avoid full table scans.

Compute profiles: Use Managed Profiles for most transforms; only request Custom Profiles for proven large-scale workloads.

Column metadata: Document output columns with descriptions in your transforms. This metadata propagates to the Ontology and is visible to end users.

README files: Every Code Repository should have a README explaining: - Purpose of the pipeline - Input datasets and their sources - Output datasets and their consumers - Transform logic overview - Known limitations or caveats

Scheduling: Align pipeline schedules with upstream data availability. Use upstream-triggered scheduling where possible to avoid polling on empty updates.

Ontology Best Practices

Primary Key: Choose a stable, immutable identifier. Avoid natural keys that may change (e.g., names). Prefer system-generated IDs or military identifiers (DODID, unit TUID, etc.).

One ontology backing dataset per object type: Do not back multiple object types from the same dataset without careful consideration — it creates complex writeback management.

Action granularity: Create targeted action types per operation. Avoid monolithic "update everything" actions that expose unintended fields.

Semantic naming: Object type names should match business domain vocabulary — use terminology that operational users understand, not technical jargon.

Shared properties: Use shared properties for cross-cutting concerns (e.g., `created_by`, `created_at`, `last_modified_at`) to ensure consistent definition.

Versioning ontology changes: Use Foundry Branching for all ontology schema changes. Never modify object types directly on Main if it will break downstream applications.

Workshop & Application Design Best Practices

Layout principles: - Max 10 visible components per view - 30–40% whitespace — do not fill every pixel - Top-level navigation: max 5 items - Follow F-shaped reading pattern (important info top-left)

Performance: - Avoid loading large object sets without filtering — use Filter variables to pre-scope queries - Use the Workshop **Performance Profiler** to identify slow widgets - Prefer Object Set Filter Variables to function calls for filtering - Enable auto-refresh sparingly — constant polling is expensive

Naming: - Module names should describe their operational purpose (not "Module 1") - Widget variables should be semantically named (`selected_soldier`, not `var1`)

User experience: - Every visible element must serve a purpose — remove decorative noise - Align filter positions consistently across all modules - Test with actual operational users before deployment

Security Best Practices

- **Never hardcode credentials**: Use environment variables or Foundry's secret management
- **Principle of least privilege**: Grant only the minimum required role; never grant Owner when Viewer suffices
- **Marking all production data**: Every dataset containing operational or sensitive data must have appropriate markings before sharing
- **CBAC for classified workloads**: Work with your Palantir Administrator to configure CBAC for SECRET and above data
- **Test on branches**: Never test schema changes or action types directly on Main with live production data
- **Parameterize all SQL**: Never use f-strings or string concatenation to build queries — always use parameterized transforms

Branching Strategy

- Branches should be **short-lived** (days, not weeks)
- One branch per feature or fix — avoid bundling unrelated changes
- Same branch name across all repositories in a product
- Create a proposal (PR) before merging — assign appropriate reviewers
- Delete (close) branches after deployment to control costs

---

28. TRAINING & CERTIFICATION RESOURCES

Official Training Portal

URL: `learn.palantir.com` (requires Foundry enrollment access)

Recommended starting point: "Speedrun: Your First End-to-End Workflow" — builds a complete pipeline + ontology + Workshop app in under 60 minutes.

Training Tracks

Track	Target Role
Data Engineer	Prepare data for reliable use in the Ontology; Pipeline Builder, transforms
Frontend & OSDK Developer	Use OSDK to build custom applications; TypeScript/Python
Application Developer	Use Workshop and Quiver to build user-facing applications
Data Analyst	Contour, Quiver, data analysis workflows
AIP / AI Developer	AIP Logic, Agent Studio, LLM integration

Certifications

- **Foundry & AIP Builder Foundations**: Entry-level badge via short quiz
- **Foundry Certification Exams**: Domain-specific exams (Data Engineering, Application Development, etc.)
- Exams have associated costs
- Contact your Palantir POC or Foundry Administrator for access

- Study guides available at `learn.palantir.com/page/exam-guides`
- **AIP Bootcamp**: Instructor-led intensive onboarding

Key Documentation URLs

Topic	URL
Foundry Docs Home	`palantir.com/docs/foundry`
Ontology Overview	`palantir.com/docs/foundry/ontology/overview`
Python Transforms	`palantir.com/docs/foundry/transforms-python/overview`
Pipeline Builder	`palantir.com/docs/foundry/pipeline-builder/overview`
Workshop	`palantir.com/docs/foundry/workshop/overview`
AIP Overview	`palantir.com/docs/foundry/aip/overview`
AIP Logic	`palantir.com/docs/foundry/logic/overview`
Agent Studio	`palantir.com/docs/foundry/agent-studio/overview`
OSDK	`palantir.com/docs/foundry/ontology-sdk/overview`
Data Lineage	`palantir.com/docs/foundry/data-lineage/overview`
Security / Markings	`palantir.com/docs/foundry/security/markings`
CBAC	`palantir.com/docs/foundry/security/classification-based-access-controls`
Foundry Branching	`palantir.com/docs/foundry/foundry-branching/overview`
Best Practices	`palantir.com/docs/foundry/building-pipelines/development-best-practices`
Platform SDK (Python)	`github.com/palantir/foundry-platform-python`

Topic	URL
OSDK TypeScript	`github.com/palantir/osdk-ts`

Community Resources

- **Palantir Developer Community**: `community.palantir.com` — Q&A, product feedback, community examples
- **build.palantir.com**: Pre-built reference applications and starter packs
- **AIP Assist**: In-platform AI assistant — ask it anything about Foundry in natural language

Third-Party Training (Unofficial)

- **Udemy**: "Prep for Palantir Foundry Data Engineering Certification" course
- **VistaSparks**: Expert Palantir Foundry training provider
- **Codestrap**: Foundry software apprenticeship programs

---

29. KEY URLS & REFERENCE INDEX

Critical Foundry API Endpoints

Operation	Endpoint
Create Transaction	`POST /api/v1/datasets/{datasetRid}/transactions`
Get Transaction	`GET /api/v1/datasets/{datasetRid}/transactions/{transactionRid}`
Get Transaction History	`GET /api/v2/datasets/{datasetRid}/branches/{branchId}/transactionHistory`
Upload File	`PUT /api/v2/datasets/{datasetRid}/files/{filePath}`
List Files	`GET /api/v1/datasets/{datasetRid}/files`

Operation	Endpoint
Get Schema	`GET /api/v2/datasets/{datasetRid}/schema`
Put Schema	`PUT /api/v2/datasets/{datasetRid}/schema`

Maven Smart System Quick Reference

Item	Detail
Program type	DEVCOM ARL contract, 5-year, up to $99.8M (2024 expansion)
Underlying platform	Palantir Foundry
EUCOM deployment	Production — covers USAREUR-AF AOR
Key tools	Foundry, Gaia, Target Workbench, Maverick, LogX
Classification	Operates at SECRET and above in operational deployments
NATO integration	SHAPE acquired MSS April 2025; JWC training underway

Foundry Security Quick Reference

Mechanism	Type	Scope
Roles (Owner/Editor/Viewer/Discoverer)	Discretionary	Resource/Project level
Organizations	Mandatory	Organization membership
Markings	Mandatory	Per-dataset, per-resource
CBAC	Mandatory	Classification hierarchy (Palantir-configured)
Object Security Policies	Mandatory	Row-level per object instance
Property Security Policies	Mandatory	Column-level per object instance

Common Foundry Developer Tasks — Quick Lookup

Task	Primary Tool	Documentation Path
Ingest data from S3	Data Connection	`data-connection/`
Build a visual pipeline	Pipeline Builder	`pipeline-builder/`
Write a Python transform	Code Repositories	`transforms-python/`
Define an object type	Ontology Manager	`object-link-types/`
Write a TypeScript function	Code Repositories (Functions)	`functions/`
Build a user application	Workshop	`workshop/`
Build a custom app (HTML/CSS)	Slate	`slate/`
Analyze data visually	Contour (datasets) / Quiver (objects)	`contour/` / `quiver/`
Train an ML model	Code Workspaces (JupyterLab)	`code-workspaces/`
Build an LLM workflow	AIP Logic	`logic/`
Build an AI assistant	AIP Agent Studio	`agent-studio/`
Build an external app	OSDK	`ontology-sdk/`
Trace data dependencies	Data Lineage	`data-lineage/`
Control data access	Markings / CBAC	`security/`
Develop safely on a branch	Foundry Branching	`foundry-branching/`

This field manual was compiled from official Palantir documentation (`palantir.com/docs/foundry`), public press releases, and platform technical references. Compiled March 2026 for USAREUR-AF Operational Data Team use. All operational security protocols from CLAUDE.md apply to this document and its use.

Sources consulted: Palantir official documentation (foundry, transforms-python, ontology, workshop, aip, agent-studio, logic, security, data-lineage, pipeline-builder, ontology-sdk, foundry-branching, building-pipelines), BusinessWire press releases, Wikipedia Project Maven, NATO SHAPE releases, C4ISRNET, Breaking Defense, DefenseScoop, Unit8 technical blog, palantir/foundry-platform-python GitHub.